AI23531 – DEEP LEARNING

EX 5 COMPARATIVE ANALYSIS OF VGG, RESNET AND GOOGLENET

AIM

To implement and compare the performance of three popular CNN architectures — VGG16, ResNet18 and GoogLeNet — for image classification using transfer learning.

PROCEDURE

1. Import PyTorch, torchvision, Matplotlib and PIL.
2. Load CIFAR-10, resize images to 224×224 and use the first 500 images split 80/20 into training and validation sets.
3. Load each pre-trained model and replace its final layer with a 10-class layer.
4. Train each model with Adam (lr = 1e-4) for 5 epochs, recording training and validation accuracy.
5. Plot the validation accuracy of the models and predict the class of a custom image.

ALGORITHM

1. Start.
2. Load and preprocess the dataset; create training and validation loaders.
3. For each model in {VGG16, ResNet18, GoogLeNet}: load ImageNet weights and replace the final fully connected layer with a 10-output layer.
4. Train for 5 epochs: forward pass, cross-entropy loss, backpropagation, Adam update; compute training accuracy.
5. Evaluate on the validation set after every epoch and store the accuracies.
6. Plot the validation accuracy of all models for comparison.
7. Predict the class of a given image with every trained model.
8. Stop.

PROGRAM

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader, random_split, Subset
import matplotlib.pyplot as plt
from PIL import Image
import os

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

```
dataset = datasets.CIFAR10(root='./data', train=True, download=True,
transform=transform)
test_dataset = datasets.CIFAR10(root='./data', train=False, download=True,
transform=transform)
dataset = Subset(dataset, range(500))
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)

def train_and_evaluate(model, name, num_epochs=5):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=1e-4)
    train_accs, val_accs = [], []
    for epoch in range(num_epochs):
        model.train()
        correct, total = 0, 0
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)
        train_accs.append(100 * correct / total)
        model.eval()
        correct, total = 0, 0
        with torch.no_grad():
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                _, predicted = outputs.max(1)
                correct += (predicted == labels).sum().item()
                total += labels.size(0)
        val_accs.append(100 * correct / total)
        print(f"{name} Epoch {epoch+1}/{num_epochs} - Train Acc: {train_accs[-
1]:.2f}%, Val Acc: {val_accs[-1]:.2f}%")
    return model, train_accs, val_accs

def get_model(name):
    if name == "vgg":
        model = models.vgg16(pretrained=True)
        model.classifier[6] = nn.Linear(4096, 10)
    elif name == "resnet":
        model = models.resnet18(pretrained=True)
        model.fc = nn.Linear(model.fc.in_features, 10)
    elif name == "googlenet":
        model = models.googlenet(pretrained=True, aux_logits=True)
        model.fc = nn.Linear(model.fc.in_features, 10)
    else:
        raise ValueError("Unknown model")
    return model

results = {}
trained_models = {}
for model_name in ["vgg", "resnet", "googlenet"]:
    print(f"\nTraining {model_name.upper()} on CIFAR-10...")
    model = get_model(model_name)
    trained_model, train_acc, val_acc = train_and_evaluate(model, model_name.upper(),
num_epochs=5)
    results[model_name] = (train_acc, val_acc)
    trained_models[model_name] = trained_model
```

```
plt.figure(figsize=(10, 6))
for name, (train_acc, val_acc) in results.items():
    plt.plot(val_acc, label=f'{name.upper()} Val Acc')
plt.title('Validation Accuracy Comparison on CIFAR-10')
plt.xlabel('Epoch'); plt.ylabel('Accuracy (%)'); plt.legend(); plt.grid(True)
plt.show()

def predict_image(image_path, models_dict):
    image = Image.open(image_path).convert('RGB')
    image = transform(image).unsqueeze(0).to(device)
    print(f"\nPrediction results for image: {image_path}")
    for model_name, model in models_dict.items():
        model.eval()
        with torch.no_grad():
            outputs = model(image)
            _, predicted = outputs.max(1)
            pred_class = class_names[predicted.item()]
            print(f"{model_name.upper():<10} => {pred_class}")

custom_image_path = "download.jpeg"
if os.path.exists(custom_image_path):
    predict_image(custom_image_path, trained_models)
else:
    print(f"\nImage not found: {custom_image_path}. Please add an image to test.")
```

OUTPUT

```
Training VGG on CIFAR-10...
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to /root/.cache/torch/hub/checkpoints/vg
g16-397923af.pth
VGG Epoch 1/5 - Train Acc: 21.25%, Val Acc: 43.00%
VGG Epoch 2/5 - Train Acc: 46.50%, Val Acc: 71.00%
VGG Epoch 3/5 - Train Acc: 73.25%, Val Acc: 61.00%
VGG Epoch 4/5 - Train Acc: 81.75%, Val Acc: 73.00%
VGG Epoch 5/5 - Train Acc: 90.25%, Val Acc: 72.00%

Training RESNET on CIFAR-10...
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints
/resnet18-f37072fd.pth
RESNET Epoch 1/5 - Train Acc: 17.75%, Val Acc: 27.00%
RESNET Epoch 2/5 - Train Acc: 91.25%, Val Acc: 53.00%
RESNET Epoch 3/5 - Train Acc: 98.50%, Val Acc: 67.00%
RESNET Epoch 4/5 - Train Acc: 99.50%, Val Acc: 73.00%
RESNET Epoch 5/5 - Train Acc: 100.00%, Val Acc: 75.00%

Training GOOGLENET on CIFAR-10...
Downloading: "https://download.pytorch.org/models/googlenet-1378be20.pth" to /root/.cache/torch/hub/checkpoint
s/googlenet-1378be20.pth

TypeError: cross_entropy_loss(): argument 'input' (position 1) must be Tensor, not GoogLeNetOutputs
```

Console output

RESULT

VGG16 reached 72% and ResNet18 reached 75% validation accuracy after 5 epochs (ResNet18 fitted the training data fastest, 100%). The GoogLeNet run stopped with a TypeError because its training-mode output (GoogLeNetOutputs, with auxiliary heads) was passed directly to the loss function; the comparison plot was therefore not produced.